Lab 4

1. Create a multiply function that accepts two numbers and returns their Product.

```
advpostgres=# SELECT multiply(5, 5);
 multiply
----------
       25
(1 row)

advpostgres=#
```

```
advpostgres=# CREATE OR REPLACE FUNCTION multiply(a NUMERIC, b NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    RETURN a * b;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

2. Create a hello_world function that takes a name as input and returns a personalized welcome message for that name.

```
advpostgres=# CREATE OR REPLACE FUNCTION hello_world(name TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN 'Welcome, ' || name || '!';
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
advpostgres=#
```

```
advpostgres=# CREATE OR REPLACE FUNCTION hello_world(name TEXT)
RETURNS TEXT AS $$
BEGIN
    RETURN 'Welcome, ' || name || '!';
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
advpostgres=# SELECT hello_world('Alaa Ahmed');
     hello_world
----------------------
 Welcome, Alaa Ahmed!
(1 row)

advpostgres=#
```

3. Create a function that accepts a number and determines whether it is

odd or even.

```
advpostgres=# CREATE OR REPLACE FUNCTION is_odd_or_even(num INTEGER)
RETURNS TEXT AS $$
BEGIN
    IF num % 2 = 0 THEN
        RETURN 'Even';
    ELSE
        RETURN 'Odd';
    END IF;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
advpostgres=#
```

```
advpostgres=# SELECT is_odd_or_even(9);
SELECT is_odd_or_even(8);
 is_odd_or_even
----------------
 Odd
(1 row)

 is_odd_or_even
----------------
 Even
(1 row)

advpostgres=#
```

4. Create a function that takes a Student ID as input and retrieves all information related to that student.

```
CREATE FUNCTION
advpostgres=# SELECT * FROM get_student_info(1);
 id |     name      |         email         | address | birth_date | gender
----+---------------+-----------------------+---------+------------+--------
  1 | Ahmed Youssef | ahmed.youssef@iti.com | Cairo   | 1999-03-15 | Male
(1 row)

advpostgres=#
```

```
advpostgres=# CREATE OR REPLACE FUNCTION get_student_info(stu_id INTEGER)
RETURNS TABLE (
    id INT,
    name TEXT,
    email TEXT,
    address TEXT,
    birth_date DATE,
    gender TEXT
) AS $$
BEGIN
    RETURN QUERY
    SELECT s.id,
           s.e_name::TEXT,
           s.email::TEXT,
           s.address::TEXT,
           s.birth_date,
           s.gender::TEXT
    FROM student s
    WHERE s.id = stu_id;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

5. Implement a function that takes the name of a subject and calculates the average grades for that subject.

```
advpostgres=# SELECT average_subject_grade('Databases');
 average_subject_grade
-----------------------
   95.5000000000000000
(1 row)

advpostgres=#
```

```
advpostgres=# CREATE OR REPLACE FUNCTION average_subject_grade(sub_name TEXT)
RETURNS NUMERIC AS $$
BEGIN
    RETURN (
        SELECT AVG(g.grade)
        FROM grades g
        JOIN subject s ON g.sub_id = s.id
        WHERE s.sub_name = average_subject_grade.sub_name
    );
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

6. Create a trigger to automatically save deleted student records from the
Student table to the Deleted_Students table.

FIRST CREATING THE TABLE Deleted_Students

```
advpostgres=# CREATE TABLE Deleted_Students (
    id INTEGER,
    e_name TEXT,
    email TEXT,
    address TEXT,
    birth_date DATE,
    gender TEXT,
    deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE
```

THE FUNCTION

```
advpostgres=# CREATE OR REPLACE FUNCTION archive_deleted_student()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Deleted_Students(id, e_name, email, address, birth_date, gender)
    VALUES (OLD.id, OLD.e_name, OLD.email, OLD.address, OLD.birth_date, OLD.gender
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

THE TRIGGER

```
advpostgres=# CREATE TRIGGER trg_archive_student
BEFORE DELETE ON student
FOR EACH ROW
EXECUTE FUNCTION archive_deleted_student();
CREATE TRIGGER
```

Students table

```
advpostgres=# SELECT * FROM student;
 id |    e_name     |         email         |  address   | track_id | birth_date | gender
----+---------------+-----------------------+------------+----------+------------+--------
  1 | Ahmed Youssef | ahmed.youssef@iti.com | Cairo      |        1 | 1999-03-15 | Male
  2 | Sara Mahmoud  | sara.mahmoud@iti.com  | Giza       |        2 | 2000-07-22 | Female
  3 | Mohamed Ali   | mohamed.ali@iti.com   | Alexandria |        3 | 1998-11-05 | Male
  4 | Nourhan Adel  | nourhan.adel@iti.com  | Mansoura   |        4 | 2001-02-17 | Female
  5 | Khaled Saeed  | khaled.saeed@iti.com  | Tanta      |        5 | 1997-09-30 | Male
(5 rows)
```

Delete the mohamed ali and check the deleted students table

```
advpostgres=# SELECT * FROM student;
 id |    e_name     |         email         |  address   | track_id | birth_date | gender
----+---------------+-----------------------+------------+----------+------------+--------
  1 | Ahmed Youssef | ahmed.youssef@iti.com | Cairo      |        1 | 1999-03-15 | Male
  2 | Sara Mahmoud  | sara.mahmoud@iti.com  | Giza       |        2 | 2000-07-22 | Female
  3 | Mohamed Ali   | mohamed.ali@iti.com   | Alexandria |        3 | 1998-11-05 | Male
  4 | Nourhan Adel  | nourhan.adel@iti.com  | Mansoura   |        4 | 2001-02-17 | Female
  5 | Khaled Saeed  | khaled.saeed@iti.com  | Tanta      |        5 | 1997-09-30 | Male
(5 rows)

advpostgres=# DELETE FROM student
WHERE e_name = 'Mohamed Ali';
DELETE 1
advpostgres=# SELECT * FROM Deleted_Students;
 id |   e_name    |       email        | address   | birth_date | gender |         deleted_at
----+-------------+--------------------+-----------+------------+--------+----------------------------
  3 | Mohamed Ali | mohamed.ali@iti.com | Alexandria | 1998-11-05 | Male   | 2025-05-01 23:14:37.239232
(1 row)
```

7. Create a trigger to monitor changes made to the student table,

including additions, updates, and deletions. This trigger will record the time of each action and provide a description of the action in another table.

The table for logs

```
CREATE TABLE Student_Audit_Log (
    log_id SERIAL PRIMARY KEY,
    student_id INTEGER,
    action_type TEXT,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    description TEXT
);
```


trigger function

```
CREATE OR REPLACE FUNCTION log_student_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO Student_Audit_Log(student_id, action_type,
description)
        VALUES (NEW.id, 'INSERT', 'New student added: ' ||
NEW.e_name);
        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO Student_Audit_Log(student_id, action_type,
description)
        VALUES (NEW.id, 'UPDATE', 'Student updated: ' || NEW.e_name);
        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO Student_Audit_Log(student_id, action_type,
description)
        VALUES (OLD.id, 'DELETE', 'Student deleted: ' || OLD.e_name);
```

```
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
advpostgres=# CREATE OR REPLACE FUNCTION log_student_changes()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO Student_Audit_Log(student_id, action_type, description)
        VALUES (NEW.id, 'INSERT', 'New student added: ' || NEW.e_name);
        RETURN NEW;

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO Student_Audit_Log(student_id, action_type, description)
        VALUES (NEW.id, 'UPDATE', 'Student updated: ' || NEW.e_name);
        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO Student_Audit_Log(student_id, action_type, description)
        VALUES (OLD.id, 'DELETE', 'Student deleted: ' || OLD.e_name);
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

the trigger

```
CREATE TRIGGER trg_log_student_changes
AFTER INSERT OR UPDATE OR DELETE ON student
FOR EACH ROW
EXECUTE FUNCTION log_student_changes();
```

```
advpostgres=# CREATE TRIGGER trg_log_student_changes
AFTER INSERT OR UPDATE OR DELETE ON student
FOR EACH ROW
EXECUTE FUNCTION log_student_changes();
CREATE TRIGGER
```

Checking the log student changes

insert

```
advpostgres=# INSERT INTO student (e_name, email, address, track_id, birth_date, gender)
VALUES ('Alaa Ahmed Gamal', 'alaa.gamal@iti.com', 'Assiut', 2, '2000-09-10', 'Male');
INSERT 0 1
```

Update

```
advpostgres=# UPDATE student
SET address = 'Cairo'
WHERE e_name = 'Alaa Ahmed Gamal';
UPDATE 1
```

## Delete

```
advpostgres=# DELETE FROM student
WHERE e_name = 'Alaa Ahmed Gamal';
DELETE 1
```

```
advpostgres=# SELECT * FROM Student_Audit_Log
WHERE description LIKE '%Alaa Ahmed Gamal%'
ORDER BY log_id;
 log_id | student_id | action_type |        action_time         |             description
--------+------------+-------------+----------------------------+-------------------------------------
      4 |          7 | INSERT      | 2025-05-01 23:24:59.52836  | New student added: Alaa Ahmed Gamal
      5 |          7 | UPDATE      | 2025-05-01 23:25:39.98707  | Student updated: Alaa Ahmed Gamal
      6 |          7 | DELETE      | 2025-05-01 23:26:12.182198 | Student deleted: Alaa Ahmed Gamal
(3 rows)
```